

The *fab* command is only one way of leveraging Fabric functionalities. The Fabric framework provides its whole API to create standalone remote execution and job orchestration tools. Using the Fabric API is the most appropriate choice in case you need to create some standalone tools, where different stages of the whole workflow are created at runtime using various cloud based APIs. For example, the Fabric API could fill the job orchestration pieces for a tool that creates cloud based resources on-the-fly, with the various steps to create cloud based environments having to be remotely executed out-of-the-box. The following code demonstrates the usage of the Fabric API by putting together a workflow that cleans up the */tmp* folder for leftover *script/zip/log* files, uploads a zip archive and unzips that to */tmp*; kicks off a bash script unzipped with *sudo* and, finally, downloads the log generated by the remote script execution:

```
from fabric.api import env, run, sudo, get, put

env.user = "<sudo USER>"
env.key_filename = "<KEY PAIR>"
env.warn_only = True
env.skip_bad_hosts = True
env.connection_attempts = 5
env.command_timeout = <TIMEOUT for REMOTE SCRIPT>
env.abort_on_prompts = True
...
env.host_string = hostname

oput = sudo('rm -f /tmp/*.zip /tmp/*.log /tmp/*.sh')
if oput.failed:
    <Handle Failure and dump str(oput)>
else:
    <Dump str(oput)>

oput = put(os.path.join(location, zipname), '/tmp')
if oput.failed:
    <Handle Failure and dump str(oput)>
else:
    <Dump str(oput)>

oput = run('unzip /tmp/%s -d /tmp' % zipname)
if oput.failed:
    <Handle Failure and dump str(oput)>
else:
    <Dump str(oput)>

oput = sudo('cd /tmp && bash %s.sh' % script)
if oput.failed:
    <Handle Failure and dump str(oput)>
else:
    <Dump str(oput)>
```

```
oput = get('/tmp/%s.log' % logfile, '.')
if oput.failed:
    <Handle Failure and dump str(oput)>
else:
    <Dump str(oput)>
```

The above examples should help you get started fast with Fabric. Fabric provides a lot of functionality, and the good detailed documentation on its site is enough to unlock its rich features.

Job orchestration through Ansible

Ansible is another option for non-centralised job orchestration. In fact, it's also Python based and automatically selects native SSH or the same SSH framework *Paramiko* on top of which Fabric is built. But Ansible is much more powerful and complicated compared to Fabric—it is a complete configuration management and job orchestrator in a single package. It can perform an easy task like running a command on remote hosts in parallel, as well as complicated tasks like bringing up and configuring cloud environments in a single shot, apart from everything and in between. Ansible uses a very simple command language based on YML, which is powerful enough to perform idempotent configuration management as well as remote job orchestration. In its configuration management capabilities, Ansible is similar to other industry standard configuration management software like Puppet and Chef, but more robust since it is non-centralised and easier due to its very simple DSL.

The prerequisites to controlling anything remotely through Ansible are SSH and Python v2.4+, which are part of almost all of the GNU/Linux based distributions. It means that you could just install Ansible on single or multiple control machines and it's all set to go. Ansible requires Python v2.6+, and the command to install it on GNU/Linux or MacOS X (Windows is not currently supported as the control machine) control machines is:

```
sudo pip install ansible
```

Now typing *ansible --help* in the console should dump a help screen mentioning all the options and arguments possible with the Ansible command line.

As I mentioned earlier, Ansible doesn't require anything like *fabfile* as in the case of Fabric, in order to start your exploration with it. But it requires an inventory file with the hostnames or IPs of the remote nodes, on which you want to run commands or scripts. The inventory file for Ansible is just a place to put down the information about the virtual/physical hosts you are planning to manage, and this information could be static or dynamic. The static inventory makes sense in case your hosts are fixed most of the time, but if you